Lab 3 report: Where Hidden Rhythms in Silent Vectors Lie

Rufina Gilmanova, COG-SUP, Comp Ling track

During this lab we constructed and trained a character-level GPT model with the hopes of observing the gradual appearance and structure of linguistic properties, more specifically the poetic properties of the English language. This lab report will explain and justify the model choice and implementation, and analyze and evaluate the generated poetry.

Section 1: Model Architecture & Design Choices

Corpus and vocabulary:

Guttenberg English poetry corpus (created by By Allison Parrish and consisting of hundereds of books from Project Guttenberg) was utilized during training. The pre-made corpus made sure to preserve line breaks, stanza boundaries and punctuation, and is often used in creative computational poetic text generation which is exactly our goal. The corpus size is 7.4MB, 7,798,473 characters. The vocabulary consists of 127 unique characters, including letters, punctuation, whitespace, and special symbols. Each character is mapped to an integer index, and the entire corpus is encoded as a sequence of these indices, resulting in a dataset of 7,798,473 tokens. The data was split into 90% training (7,018,625 tokens) and 10% validation (779,848 tokens), ensuring that model performance could be monitored on unseen data during training.

Overview of the Model :

- It's a decoder-only Transformer that predicts the next character given the previous ones
- It operates at the character level, which keeps the vocabulary small

- Implementation includes standard components: embeddings, attention, feed-forward blocks, residual connections, layer normalization

Architecture:

1. **Token embeddings**: Each of the 82 unique characters (letters, punctuation, whitespace) is mapped to a 256-dimensional embedding vector. This allows the model to learn rich representations of characters.
2. **Positional embeddings**: We use learned positional embeddings (one embedding vector per position up to context length 256). This is simpler than sinusoidal encodings and works well for fixed-length contexts like ours. An alternative would be rotary positional embeddings (RoPE), which are more efficient, but we prioritized simplicity.
3. **Attention mechanism**: "We implement multi-head causal self-attention using `torch.nn.functional.scaled_dot_product_attention` with `is_causal=True`. This function automatically uses FlashAttention when available (as was recommended), which is faster and more memory-efficient than a traditional masked attention implementation. We use 8 attention heads, each with head dimension 32 (256 / 8).
4. **Feed-forward blocks:** Each transformer block includes a feed-forward MLP with 4× expansion: the hidden layer has dimension 4 × 256 = 1024. This is followed by ReLU and another linear layer back to 256.
5. **Layer structure:** We stack 6 transformer blocks. Each block applies: LayerNorm → MultiHeadAttention → Residual, then LayerNorm → FeedForward → Residual (pre-norm formulation).
6. **Output projection:** After all transformer blocks and a final layer normalization, we project to the vocabulary size (82) using a linear layer. This produces logits over all possible next characters.

Design Justifications

Context length = 256 characters:

- A context of 256 characters corresponds roughly to 30–50 words in English poetry. This length is long enough to capture relationships between lines

(rhyme patterns, stanza structures) while remaining computationally tractable on Colab. Shorter contexts (e.g., 128) might miss rhyme pairs separated by a line break; longer contexts (e.g., 512) would increase memory and training time significantly.

Training pairs:

- For each position i in the corpus, we create a training example with input x = characters[i:i+256] and target y = characters[i+1:i+257]

Hyperparameters:

- This configuration (suggested by the instructions) provides a balance. 6 layers give the model enough expressiveness to learn poetic patterns without being so deep that training becomes unstable. 256 dimensions strikes a balance between model capacity and GPU memory (I however still kept running into the 'out-of GPU' message). 8 heads allow multiple attention patterns to run in parallel (some might attend to rhyme patterns, others to meter or line breaks).

```
CONFIG = {
    'vocab_size':      vocab_size,
    'embed_dim':       256,
    'num_heads':       8,
    'num_layers':      6,
    'context_length':  256,
    'dropout':         0.1,
    'batch_size':      64,
    'learning_rate':   3e-4,
    'max_steps':       20000,
    'eval_interval':   500,
    'sample_interval': 2000,
}
```

Learned positional embeddings over sinusoidal:

- We chose learned positional embeddings for simplicity and because our context length is fixed (256). Learned embeddings are easier to interpret—each position gets its own embedding vector that the model learns. Sinusoidal

encodings would be more efficient and extrapolate better to longer sequences, but for our fixed-length setting, learned embeddings should be sufficient.

Attention implementation choice:

- We use `torch.nn.functional.scaled_dot_product_attention` with `is_causal=True`, which is more efficient than manual masked attention. The function automatically dispatches to FlashAttention when the GPU supports it, reducing memory usage and training time by roughly 2–3×. This is alligns with the recommendations in instructions to the lab.

Section 2. Training Configuration & Training Dynamics & Learning Curves

Training Configuration:

The model follows the suggested configuration: embed_dim 256, 8 attention heads, 6 transformer layers, context length 256, dropout 0.1, batch size 64, learning rate 3×10^{-4}, AdamW optimizer, trained for 20,000 steps. The context length of 256 characters can capture roughly 2–4 lines of verse simultaneously, which is sufficient to model intra-stanza rhyme and rhythm, though cross-stanza dependencies remain out of reach. The resulting model has 4,865,151 parameters (~18.6 MB).
Monitoring was conducted every 500 steps for both train and validation loss (with BPC reported alongside), and text samples were generated every 2,000 steps.

Sample progression analysis

Step 2000 | train loss: 1.8695 (bpc: 2.697) | val loss: 1.8899 (bpc: 2.727) | time: 718s

-- Sample at step 2000 ---

Sappan refers, with now selfull Same the richt;
New a sirt the my string stray their cansming,
O Caria, the were, waken, is this steed.
And nightle sprop and the leap, thou voice thiness bare,
Be of t

Step 2000 (BPC ~2.70): The model has learned basic English orthography: most tokens are plausible letter combinations, but word-level semantics are absent. Neologisms dominate: *selfull*, *cansming*, *nightle*, *sprop*, *thiness*. Line structure is superficially present (capitalisation at line starts, semicolons, commas) but syntactic coherence within lines is weak. The model has internalized that poetry involves punctuation and line breaks, but has not yet learned which words actually exist or how they combine grammatically.

Step 12000 | train loss: 1.3918 (bpc: 2.008) | val loss: 1.5182 (bpc: 2.190) | time: 4344s

-- Sample at step 12000 ---

Far intendent Springe, prayer hie,
Water heere how transported his stone,
And children at the blind and the earth,
On her head in windy sun,
And hear a man the wad right hand,
Her fair eyes looked as

Step 12000 (BPC ~2.19): A marked qualitative leap. Real English words now predominate —(*children, earth, windy, prayer, stone, hand),* and noun phrases are syntactically plausible (*blind and the earth, windy sun, fair eyes*). Occasional archaisms (*heere, hie, wad*) reflects that the model is capturing period-specific poetic register from the Gutenberg poetry corpus. Lines are still loosely connected semantically and the opening (*Far intendent Springe*) still contains a neologism, but the overall impression is of poorly structured but recognisable verse rather than noise.

Step 20000 | train loss: 1.3252 (bpc: 1.912) | val loss: 1.4954 (bpc: 2.157) | time: 7263s

-- Sample at step 20000 ---

Then Carry bore himself I was clearly tilled.
Higher left, higher fled along the moan,
Stationed to lion's limbs of the sky,

His banquet spoused or bone a prince in his son.
Thus we hear; he grieved a

Step 20000 (BPC ~1.91): The best structural coherence so far. Lines with a consistent rhythm, verbs are correctly inflected (*bore, fled, grieved, spoused*), and there is even an epic register (*lion's limbs, banquet, prince, rampart*) consistent across multiple lines. Neologisms are rare (*spoused* being the main instance, maybe a blend of *espoused* and *spouse?*). Discourse-level connectives appear (*Then, Thus*), suggesting the model has learned something about narrative sequencing within a stanza, not just local character patterns.

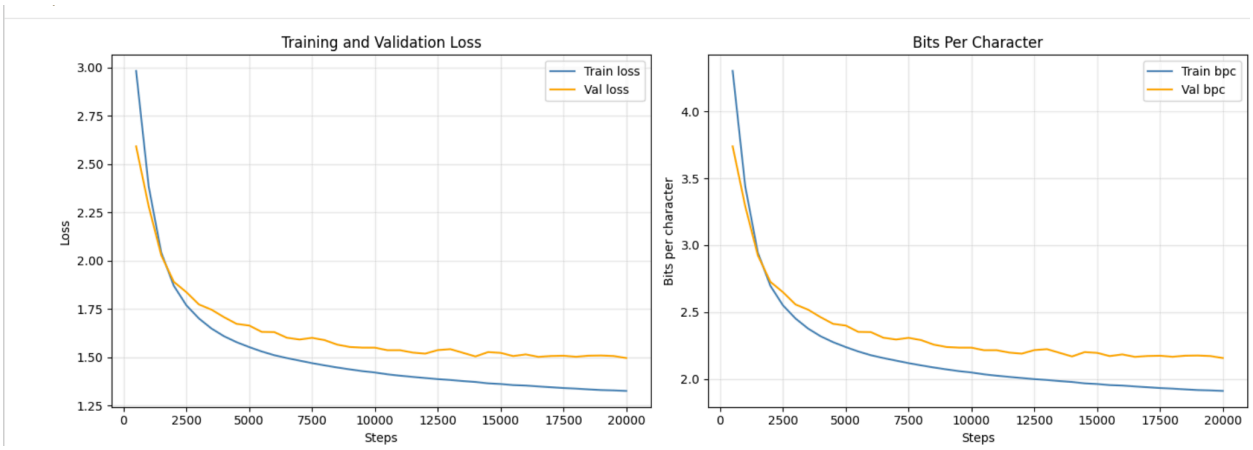
Overall trajectory: I have picked these 3 samples to showcase progression of learning and how it affects the generated poetry. The model moves through distinguishable phases: character-to-word (steps 0–4000, steep loss drop), word-to-syntax (steps 4000–12000, moderate improvement), and syntax-to-register (steps 12000–20000, slow refinement). The train–val BPC gap widens across these phases (essentially zero at step 1500, ~0.24 by step 20000), reflecting the model beginning to overfit fine stylistic details of the training corpus. This is also visible in the generated text: the late sample's epic diction closely mirrors the vocabulary distribution of classical English poetry corpora, indicating the model is memorizing stylistic patterns rather than generalizing.

To solve the overfitting problem a potential solution would be increasing dropout (from 0.1 to 0.3, this is addressed later on), adding weight decay to AdamW, and most importantly using a bigger corpus. At 7.4 MB the corpus is on the smaller end. A 15–20 MB corpus could give the model more variation to generalize from patterns. Early stopping based on validation loss could also help.

You can also observe the abrupt endings of the lines at the end of the stanzas: this is a context window truncation effect. The model generates up to the 256-character context limit and simply stops mid-sequence as no explicit end-of-poem token or stopping criterion was implemented, so generation is cut off mechanically. Adding a dedicated end-of-stanza or end-of-poem token to the

vocabulary and training the model to predict it would allow generation to terminate naturally.

Showing the curves:



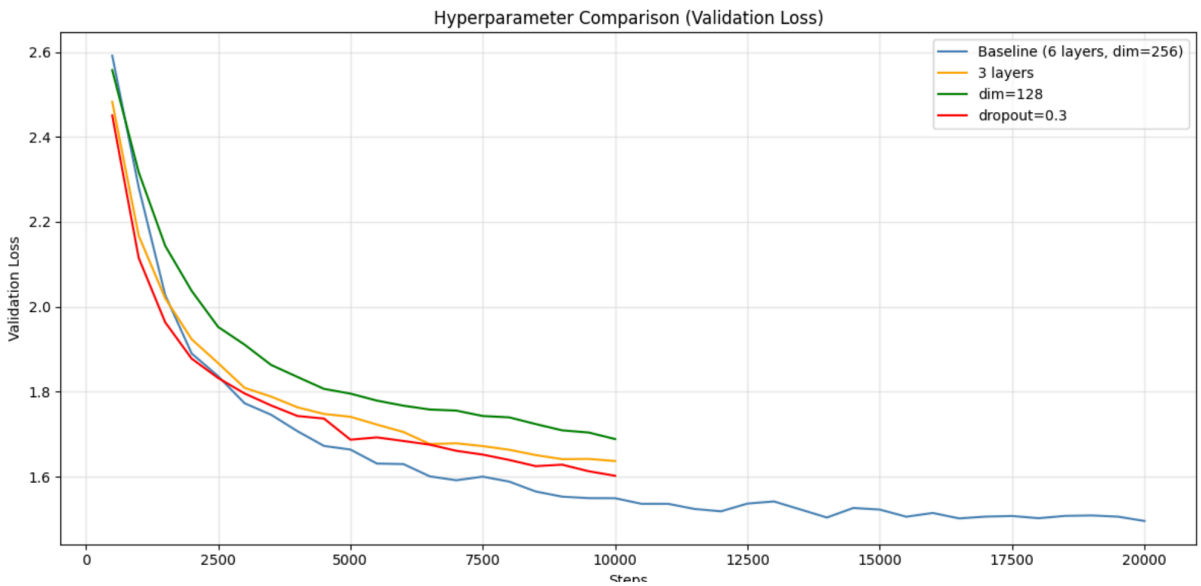
We can observe that Training Val Loss curve declines quickly up to step 5000, rapid loss reduction in the early steps is driven by the model learning basic character and word statistics. Which is then followed by slower returns as it refines higher-level stylistic patterns; and then somewhat plateaus towards the very end. The BPC val curve declines rapidly before step 4000 (from 3.74 to ~2.46), and then the curve declines less steeply.

Training approximately converged by step 20,000, with both train and validation loss stabilizing (train BPC 1.91, val BPC 2.16) and changes of less than 0.02 per 500 steps in the final phase (additional steps may have brought marginal gains).

Section 3. Hyperparameters discussion

As was suggested by the instructions I have explored hyperparameters variance. I have chosen to discuss the effect of fewer layers (3 vs 6), smaller model (embed_dim 128, 4 heads), and higher dropout (0.3) on the model's performance.

Plot:



Interpretations:

Fewer layers (3 vs 6).

Competitive. By step 10000 the model reaches val loss 1.637, only marginally worse than the baseline's val loss at the same point. The train-val gap is also narrower (~0.13 vs ~0.22 for baseline), indicating less overfitting. The shallower model learns more slowly in early steps but catches up efficiently, suggesting that for a 7.4 MB poetry corpus, 6 layers may be a little too much and the additional depth primarily contributes memorization rather than generalisation. The trade-off is roughly half the representational capacity for similar validation performance and better regularization.

Smaller model (embed_dim 128, 4 heads). The weakest variant. Val loss at step 10000 is 1.688, notably higher than both baseline and fewer_layers. The train-val gap (~0.10) is the tightest of all variants, but this reflects underfitting rather than good generalization: the model simply lacks capacity to overfit. Early steps show slower convergence than fewer layers despite similar parameter reduction, suggesting that embedding dimension is a more critical bottleneck than depth for character-level modelling.

High dropout (0.3). The most interesting result. Early training is slower (val 2.44 at step 500 vs 2.59 baseline, but train loss also higher due to dropout noise), yet

by step 7500 the train-val gap is only ~0.11, the tightest among full-capacity models. This directly addresses the memorization concern raised earlier: stronger dropout forces the model to learn more robust, distributed representations. The cost is slower convergence. However given more steps, this variant would probably generalize better than the baseline.

In conclusion, fewer layers offers the best efficiency-to-performance ratio for this corpus size. High dropout is the most principled remedy for overfitting but needs more training steps to fully pay off.

Offering unbacked linguistic intuition that I think would help in improving the model's performance by adjusting the following:

Longer context (512+) would allow the model to see across stanza boundaries and learn rhyme schemes, which operate at distances beyond 256 characters. Judging by the samples, the model currently cannot "remember" the end of line 1 when generating the end of line 4.

Special tokens. As was mentoined earlier, explicitly marking line breaks with a dedicated token and stanza boundaries with another. This would give the model a cleaner signal for poetic form, teaching it the grammar of verse layout explicitly rather than expecting it to infer from whitespaces.

The core tension here is that poetry operates simultaneously at multiple structural levels: individual characters form syllables, syllables form words, words form metrical feet, feet form lines, lines form stanzas. A character-level model must learn all of these hierarchies from scratch, so depth matters more than width. This is why fewer layers hurt less than a smaller embedding: three layers can still learn characters→words→lines, but a narrow embedding cannot represent enough distinct linguistic contexts simultaneously.

Section 4. Generated Poems Analysis.

This section offers the analysis of generated poems varied by different temperatures.

Generated poem 1, temperature 0.5 (conservative)

> And the deep is strong and the struggling boat,
> Which the great crowds is dead, and the slaves
> And the shining rain that shines and states
> For the land of the air is running from the states
> And the third states and the storm of the states.
> A little shoulder stands in the stately door,
> And the lonely princess of the shining states
> Where we shall not be the fair and the farmer states;
> The ships whic

The deterministic model. Produces grammatical stability but exposes a repetition problem. The word *states* appears 7 times across eight lines, and the conjunction *And* opens five consecutive lines. The model locks onto high-probability tokens and loops. It seems to have learned the syntactic frame of English verse (*And the X of the Y*) but applies it mechanically without semantic progression. The imagery (*shining rain, lonely princess, struggling boat*) is individually coherent but with no narrative or emotional direction. This reflects the model's inability at this temperature to escape locally optimal but globally repetitive patterns.

Generated poem 2, temperature 1.0 (balanced)

> Judgment of EPINUO-COFT, of all sense
> And all-day, on that Ilium nose
> Springs, who from left Arice his champain bore
> Arise, who to Argos feet, son of Jove,
> Had arm'd the Hector to my fathers shun.
> Ah, here among these Trojans! the royal found
> Who cry, or whooping in battle sound?
> Why supplied: the other body thus?
> Here haughty-one I felt alone? Could to the last;
> Here oft a doubtless day; with the

The most natural sounding variant, output is poetically interesting. The epic register is sustained across the full sample (*Trojans, Hector, Argos, Ilium, Jove),* indicating the model has learned to maintain a stylistic domain when sampling is

balanced. Rhyme emerges spontaneously (*bore/Jove, sound/found, thus/last*), suggesting the model has internalized some end-of-line phonological patterning. Neologisms remain (*EPINUO-COFT*) but are rare. Syntactically, lines are well-formed and discourse connectives (*Ah, Why, Here*) create a dramatic voice. This is the temperature where learned structure and creative variation are best balanced.

Generated poem 3, temperature 1.3 (creative)

> I disley,--Copious! Yes, my case.
> Wax now? Well; nit tip! 106.
> A gimmy tink? the Queen, in verse! now! as'a]
> qUickly woo, angels anyus?
> Le
> Hath am nought-happy burthings compliant._
> Arbatrary around her silent flame._"
> like a lass,
> They arm'd herself, rich in rattling degree
> She will turn by a rose by rings come, and stretching out forth
> "The mysterious nation--Giverwood be!
> Eig! A custom-tinkling

Mostly noise and gibberish but let’s dissect. Punctuation becomes erratic (*Wax now? Well; nit tip! 106*), invented words in abundance (*disley, gimmy, anyus, burthings*), numeric tokens appear, suggesting the model is sampling from the long tail of its character distribution. However, fragments of coherence still surface (*silent flame, stretching out forth, rich in rattling degree, the mysterious nation*), revealing that poetic register is still encoded. The output resembles an avant-garde verse rather than classical poetry: structure degrades gracefully rather than catastrophically.

Concluding, temperature 1 seems to be the perfect trade-off between surprise and structure.

Section 5. Overall Conclusions

Overall, we can see that the model acquired linguistic structure in a clear hierarchy. Character-level statistics were learned almost immediately, valid English orthography appeared within the first few hundred steps. Word-level patterns followed by step 2000, with recognizable vocabulary and basic morphology, though neologisms remained frequent. By step 12000, syntactic frames were stable and a consistent poetic register (epic diction, archaic vocabulary, classical proper nouns) was maintained across entire stanzas. End-of-line patterns suggestive of rhyme appeared spontaneously at temperature 1.0, indicating the model learned something about phonological regularity at line boundaries without any explicit supervision.

Where the model fell short:

Cross-stanza coherence never emerged, which is a direct consequence of the 256-character context window, the model cannot hold a rhyme scheme or thematic argument across more than two or three lines simultaneously. Semantic progression within a poem remains absent; lines are locally plausible but globally directionless and lack any kind of narration. Repetition under low temperature exposes that the model has learned syntactic frames as statistical attractors rather than as generative structures with meaningful variation.

On design choices:

Depth proved more important than width. Fewer layers hurt less than a smaller embedding dimension, consistent with the intuition that each layer captures a different level of linguistic abstraction. High dropout showed to be the most principled intervention against overfitting, narrowing the train-val gap more effectively than architectural reduction, at the cost of slower convergence. The 256-character context was, in my opinion, the most limiting architectural decision. It is adequate for word and line level learning but categorically prevents the model from acquiring stanza-level structure.

What was surprising

The most unexpected finding was how competitive the three-layer model was against the six-layer baseline. Given that poetry requires capturing hierarchical structure across multiple scales, the expectation would be that depth is critical, yet halving the layers produced only marginal validation loss degradation while meaningfully reducing overfitting. This suggests the corpus itself, rather than model capacity, is the primary bottleneck. Equally surprising was the robustness of poetic register under high temperature: even at 1.3, fragments of epic diction survived amid near-random character sampling, indicating that stylistic register is among the most deeply encoded features in the model's representations.